BTS SIO – OPTION SLAM

Marchand Super Admin

Documentation Technique de l'Application Lourde – Épreuve E6

Application desktop de gestion développée avec .NET 9 et Avalonia UI, utilisant le pattern MVVM pour superviser les données de la marketplace Laravel via API REST.

Projet	Site Test Marchand – Application Desktop
Framework UI	Avalonia UI (XAML)
Langage / Runtime	C# / .NET 9.0
Pattern	MVVM (CommunityToolkit.Mvvm)
Date du document	Avril 2026

Option BTS	SIO – SLAM

Sommaire

1. Architecture et Stack Technologique
2. Structure du Projet (MVVM)
3. Flux de Données et Communication Réseau
4. API Backend Laravel Utilisée
5. Sécurité et Prérequis
6. Compilation et Déploiement

1. Architecture et Stack Technologique

1.1 Technologies Utilisées

Composant	Technologie	Version
Langage Core	C#	13.0
Framework de base	.NET SDK	9.0
Interface Graphique	Avalonia UI	11.3.9
Architecture Logicielle	CommunityToolkit.Mvvm	8.2.1
Communication Réseau	HttpClient (System.Net.Http)	Natif .NET
Sérialisation	System.Text.Json	Natif .NET

1.2 Pourquoi Avalonia UI ?

Avalonia a été choisi comme alternative à WPF car il permet une compilation **multiplateforme** (Windows, macOS, Linux) tout en conservant la puissance du XAML et du databinding natif de l'écosystème .NET.

2. Structure du Projet (MVVM)

Le projet est organisé de manière stricte selon le pattern Model-View-ViewModel pour découpler l'interface de la logique métier.

2.1 Fichiers d'initialisation et configuration

- `App.axaml` : Thème global, dictionnaires de ressources et point d'entrée de l'interface graphique.
- `App.axaml.cs` : Logique d'initialisation de l'application et définition de la vue principale.
- `ViewLocator.cs` : Résolution automatique des Vues (Views) en fonction de leurs ViewModels respectifs via les DataTemplates.

2.2 Dossier `/Models`

Contient les entités Data Transfer Object (DTO) qui reflètent le format JSON renvoyé par le backend Laravel.

- `AdminUser.cs` : Représente un inscrit de la plateforme.
- `AdminConnectionLog.cs` : Représente un journal de connexion.
- `UpgradeRequestItem.cs` : Représente une demande de passage en entreprise avec infos SIRET.

2.3 Dossier `/Services`

Contient la couche d'accès aux données.

- `DesktopAdminApiClient.cs` : **Classe instanciée** utilisant une propriété `HttpClient` statique (`static readonly`). Elle s'occupe de construire les URL, d'injecter le header d'authentification (`X-Desktop-Admin-Token`), de parser les JSON et de lancer des exceptions typées en cas d'erreur HTTP.

2.4 Dossier `/ViewModels`

Le cœur du réacteur applicatif.

- `ViewModelBase.cs` : Hérite de `ObservableObject` (CommunityToolkit).
- `MainWindowViewModel.cs` :
  - Contient les listes observables (`ObservableCollection<T>`) pour mettre à jour l'UI en temps réel.
  - Contient les propriétés bindées à la vue (ex: `SelectedUpgradeRequest` , `RejectionReason`) avec l'attribut `[ObservableProperty]` .
  - Déclare les `IAsyncRelayCommand` pour l'exécution des requêtes asynchrones (ex: `LoadUsersCommand` , `ApproveUpgradeRequestCommand`).

- Gère l'interface de détails utilisateur avec la modale : `SelectedDetailUser` pour stocker les données de l'utilisateur ciblé, `ShowUserDetailsCommand` pour afficher la modale, et `CloseUserDetailsCommand` pour la masquer.

2.5 Dossier `/Views`

L'interface graphique, déclarative.

- `MainWindow.axaml` : Code XAML divisé en onglets (TabControl), grilles, et composants de liste. Le databinding (ex: `Command="{Binding ApproveUpgradeRequestCommand}"`) connecte directement les boutons au ViewModel. Contient également l'overlay (modale) pour afficher les détails complets d'un utilisateur sélectionné.
- `MainWindow.axaml.cs` : Le code-behind de la vue principale, gérant l'initialisation des composants UI (méthode `InitializeComponent()`).

3. Flux de Données et Communication Réseau

Voici le cycle de vie complet d'une action, par exemple le refus d'une demande vendeur :

```
┌──────────────┐       ┌──────────────────────┐       ┌──────────────────────┐
│ UI (AXAML)   │──────▶│ MainWindowViewModel  │──────▶│ DesktopAdminApiClient │
│ Clic Refus   │       │ Execute RejectCommand │       │ POST /api/../reject   │
└──────────────┘       └──────────────────────┘       └──────────────────────┘
                                ▲                              │
                                │                              ▼
                      ┌────────────────┐              ┌────────────────┐
                      │ Mise à jour    │◀─────────────│ API Backend    │
                      │ ObservableCol  │              │ (Laravel)      │
                      └────────────────┘              └────────────────┘
```

Grâce au système d'**ObservableProperty**, dès que le ViewModel supprime la demande de sa liste interne après succès de l'API, l'interface graphique (ListBox) retire l'élément sans qu'aucune manipulation d'UI ne soit nécessaire en C#.

4. API Backend Laravel Utilisée

L'application lourde consomme un sous-ensemble privé de l'API Laravel, protégé par le header `X-Desktop-Admin-Token`.

Méthode	Endpoint	Action métier
`GET`	`/api/desktop-admin/ping`	Vérifier la validité du Token
`GET`	`/api/desktop-admin/users`	Récupérer la liste de tous les utilisateurs
`PATCH`	`/api/desktop-admin/users/{id}`	Mettre à jour le rôle (client/entreprise/admin)
`DELETE`	`/api/desktop-admin/users/{id}`	Supprimer un utilisateur
`GET`	`/api/desktop-admin/logs`	Récupérer les journaux de connexion
`GET`	`/api/desktop-admin/upgrade-requests`	Récupérer les demandes de vendeurs en attente
`POST`	`/api/desktop-admin/upgrade-requests/{id}/approve`	Approuver la demande d'un vendeur
`POST`	`/api/desktop-admin/upgrade-requests/{id}/reject`	Refuser la demande (nécessite `reason`)

5. Sécurité et Prérequis

5.1 Sécurité des communications

Toute la communication entre l'application lourde Avalonia et le backend Laravel est sécurisée par une authentification par token API. Le `DesktopAdminApiClient` injecte systématiquement ce jeton d'accès dans les en-têtes HTTP de chaque requête via le header spécifique `X-Desktop-Admin-Token`. Les requêtes non authentifiées ou avec un jeton invalide sont immédiatement rejetées par le serveur.

5.2 Prérequis système

Pour le développement, l'exécution en mode source ou la compilation de l'application, l'environnement nécessite l'installation du **SDK .NET 9.0**. Une fois compilée et déployée de manière autonome (*Self-Contained*), l'application peut être exécutée sur la machine cible sans que l'utilisateur n'ait besoin d'installer le framework .NET.

6. Compilation et Déploiement

6.1 Environnement de développement

```
dotnet run
```

Compile le projet en mode `Debug` et lance l'interface instantanément.

6.2 Génération pour la production (Release)

Pour générer un fichier binaire optimisé pour la machine courante :

```
dotnet publish -c Release
```

6.3 Cross-Compilation Single File

Pour créer un unique fichier exécutable (ex: `.exe` sous Windows) incluant le runtime .NET (Self-Contained) de sorte que l'utilisateur n'ait rien à installer :

```
dotnet publish -c Release -r win-x64 --self-contained true -p:PublishSingleFile=true
```

Remplacer `win-x64` par `osx-x64`, `osx-arm64`, ou `linux-x64` selon la plateforme ciblée.